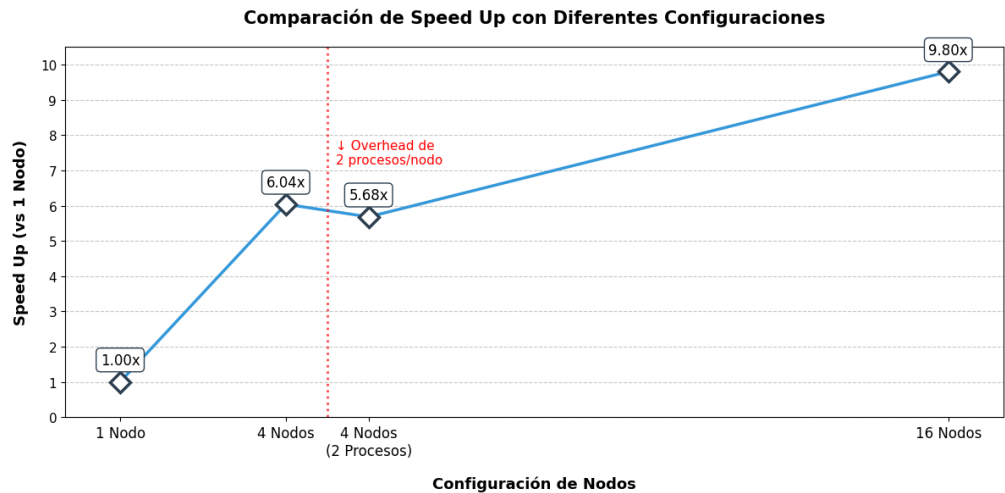


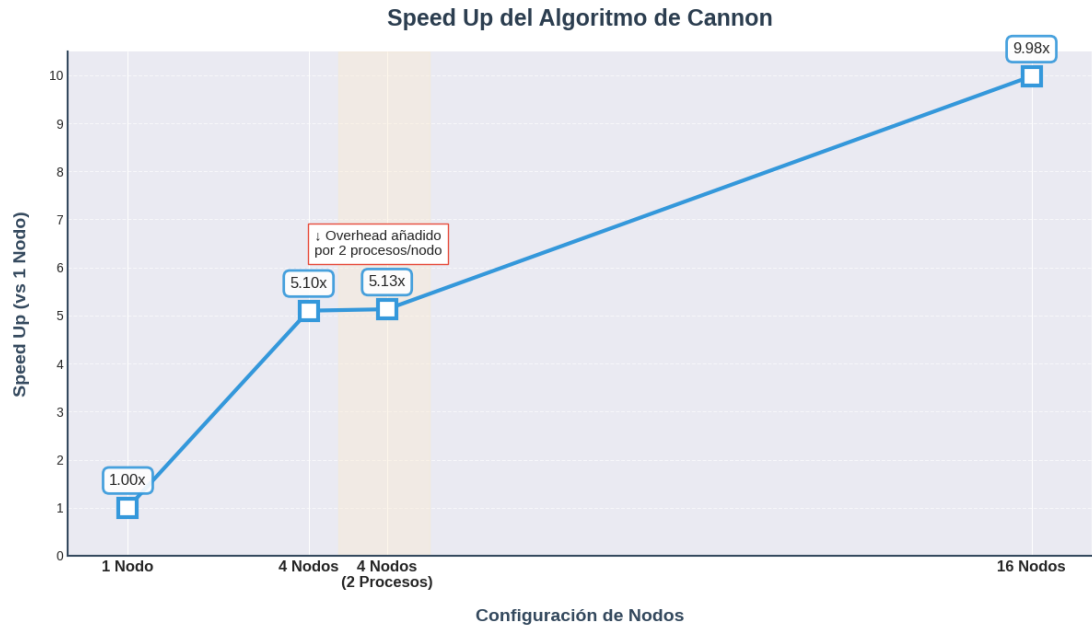
ALGORITMO DE FOX

Métrica	1 Nodo	4 Nodos	4 Nodos (2procesos)	16 Nodos
Tiempo medio	11.604138	1.926094	2.042579	1.18907
Tiempo más alto	12.887646	2.154105	2.107730	1.251741
Tiempo más bajo	10.477805	1.790961	1.910982	1.148158
Variación	2.409841	0.363144	0.196748	0.103583
Speed up	1.0	6.04	5.68	9.8



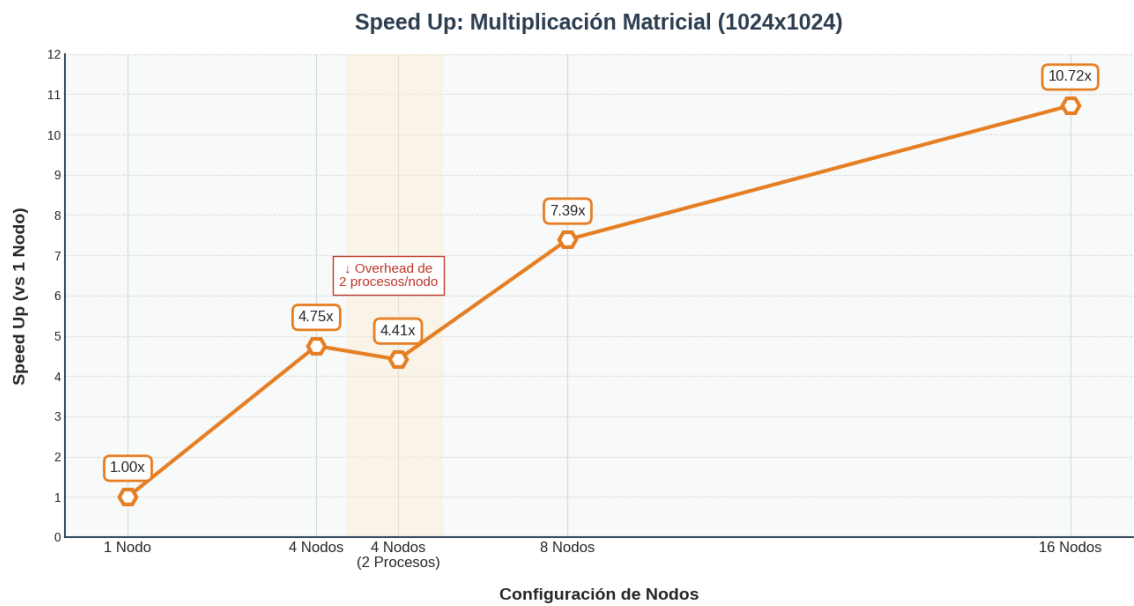
ALGORITMO DE CANNON

Métrica	1 Nodo	4 Nodos	4 Nodos (2procesos)	16 Nodos
Tiempo medio	10.740777	2.114	2.092756	1.076616
Tiempo más alto	11.789626	2.234247	2.192979	1.180969
Tiempo más bajo	8.009808	1.881399	1.992775	0.945468
Variación	3.779818	0.352848	0.200204	0.235501
Speed up	1.0	5.1	5.13	9.98



ALGORITMO DE BLOCK ROW-WISE

Métrica	1 Nodo	4 Nodos	4 Nodos (2procesos)	8 Nodos	16 Nodos
Tiempo medio	7.424116	1.568753	1.682913	1.004366	0.692126
Tiempo más alto	8.581758	1.675503	2.000700	1.083501	0.705126
Tiempo más bajo	6.701251	1.453765	1.480423	0.890586	0.682067
Variación	1.880507	0.221738	0.520277	0.192915	0.023059
Speed up	1.0	4.75	4.41	7.39	10.72



TIEMPOS MEDIOS DE ALGORITMOS DE MULTIPLICACIÓN

Algoritmos	1 Nodos	4 Nodos	4 Nodos (2 procesos)	8 Nodos	16 Nodos
Fox	11.604138	1.926094	2.042579	-	1.18907
Cannon	10.740777	2.114	2.092756	-	1.076616
Block row-wise	7.424116	1.568753	1.682913	1.004366	0.692126

Algoritmo	Ventajas	Desventajas
Strassen	• Reduce la complejidad teórica de la multiplicación ($\approx O(n^{2.81})$ vs $O(n^3)$) • Puede ser más rápido para matrices grandes	• Mayor complejidad en la implementación distribuida • Comunicación intensa • Más memoria requerida • No tan eficiente para matrices pequeñas o bloques muy pequeños
Cannon	• Buena escalabilidad con procesos organizados en malla 2D • Comunicación estructurada y predecible • Balance de carga homogéneo	• Requiere número de procesos como cuadrado perfecto • Coste inicial de alineación • No aprovecha bien arquitecturas no regulares
Fox	• Similar a Cannon pero con mejor paralelismo en la comunicación • Más flexible que Cannon para ciertas arquitecturas	• También requiere cuadrado perfecto de procesos • Más complejidad en la gestión de comunicación • Coste adicional en desplazamiento inicial
Block Row-wise	• Muy simple de implementar • Escala bien con número creciente de procesos • Menos sobrecarga de sincronización	• Comunicación no optimizada (broadcast para bloques completos) • Menos eficiente en escalabilidad fina • Puede generar cuellos de botella en comunicación

- **Algoritmo de Cannon:** <https://iq.opengenus.org/cannon-algorithm-distributed-matrix-multiplication/>
- **Algoritmo de Fox:** <https://www.youtube.com/watch?v=ig43BZI4ybw>
- **Algoritmo de Strassen:** <https://www.youtube.com/watch?v=sZxjuT1kUd0>

De aquí también nos ayudo a entender ciertos aspectos:

1. Dongarra, J. et al. (1998). An Introduction to Grid and Block Algorithms for Matrix Multiplication. LAPACK Working Note 120.

<https://netlib.org/lapack/lawnspdf/lawn120.pdf>

2. Quinn, M. J. (2004). Parallel Programming in C with MPI and OpenMP. McGraw-Hill.

3. Grama, A., Gupta, A., Karypis, G., Kumar, V. (2003). Introduction to Parallel Computing. Addison Wesley.

4. Van de Geijn, R., & Watts, J. (1997). SUMMA: Scalable Universal Matrix Multiplication Algorithm. Concurrency: Practice and Experience.

5. Strassen, V. (1969). Gaussian elimination is not optimal. Numerische Mathematik, 13, 354–356.

6. Gropp, W., Lusk, E., & Skjellum, A. (1999). Using MPI: Portable Parallel Programming with the Message Passing Interface. MIT Press.